

МИНОБРНАУКИ РОССИИ
САНКТ-ПЕТЕРБУРГСКИЙ ГОСУДАРСТВЕННЫЙ
ЭЛЕКТРОТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ
«ЛЭТИ» ИМ. В.И. УЛЬЯНОВА (ЛЕНИНА)
Кафедра МО ЭВМ

ОТЧЕТ
по Индивидуальному домашнему заданию
по дисциплине «Введение в нереляционные базы данных»
Тема: Сервис для электронных таблиц и форм

Студенты гр. 3341

Шуменков А.П.
Анисимов Д. А.
Игнатьев К.А.

Преподаватель

Заславский М.М.

Санкт-Петербург

2026

ЗАДАНИЕ

Студенты

Шуменков А.П.

Анисимов Д. А.

Игнатъев К.А.

Группа 3341

Тема проекта: Сервис для электронных таблиц и форм

Исходные данные: Давайте замахнемся на создание нашего ответа GoogleSheets, используя СУБД(ArangoDB)

Содержание пояснительной записки:

1. Введение
2. Назначение и возможности приложения
3. Качественные требования к решению
4. Сценарии использования
5. Модель данных
6. Разработанное приложение
7. Вывод
8. Приложение
9. Используемая литература
10. Примеры запросов

Предполагаемый объем пояснительной записки:

Не менее 10 страниц.

Дата выдачи задания: 16.01.2026

Дата сдачи реферата: 22.05.2026

Дата защиты реферата: 22.05.2026

Студенты гр. 3341

Шуменков А.П.

Анисимов Д. А.

Игнатъев К.А.

Преподаватель

Заславский М.М.

АННОТАЦИЯ

В работе рассматривается проект веб-приложения, объединяющего функциональность электронных таблиц и форм. Пользователь может регистрироваться и входить в систему, создавать таблицы, редактировать ячейки, применять фильтры, создавать формы, собирать ответы, переносить результаты в таблицы, просматривать историю действий, выполнять импорт, экспорт и строить статистику. Основное внимание уделено модели данных: для предметной области предложена NoSQL-модель в ArangoDB.

Найти исходный код и всю дополнительную информацию можно по ссылке:
<https://github.com/moevm/nsql1h26-table>

ANNOTATION

The paper discusses a web application project that combines the functionality of spreadsheets and forms. Users can register and log into the system, create spreadsheets, edit cells, apply filters, create forms, collect responses, transfer results to spreadsheets, view action history, perform import and export, and generate statistics. The main focus is on the data model: a NoSQL model in ArangoDB is proposed for the subject area.

The source code and all additional information can be found at the link:
<https://github.com/moevm/nsql1h26-table>

Оглавление

1. Введение
2. Назначение и возможности приложения
3. Качественные требования к решению
4. Сценарии использования
5. Модель данных
6. Разработанное приложение
7. Вывод
8. Приложение
9. Используемая литература
10. Примеры запросов

1. Введение

Цель проекта - разработать систему для работы с пользовательскими таблицами и формами, близкую по идее к связке Google Sheets и Google Forms. Приложение должно хранить динамические структуры данных, поддерживать быстрый переход между списками сущностей, редактированием, просмотром ответов и аналитикой. Для такой предметной области заранее фиксированная схема является ограничением: набор колонок таблицы, поля формы, варианты ответов и состав статистики могут отличаться от объекта к объекту.

В рамках работы были описаны пользовательские сценарии, построены две модели данных - нереляционная и реляционная, приведены примеры данных и запросов, а также выполнено сравнение моделей по объему хранения и количеству операций в типовых сценариях.

2. Назначение и возможности приложения

Система предназначена для хранения и обработки табличных данных, сбора пользовательских ответов через формы и получения простой статистики по накопленным данным. Основными объектами приложения являются пользователь, таблица, строка таблицы, форма, поле формы, ответ на форму, журнал действий и связи доступа.

- авторизация и регистрация пользователей;
- создание, просмотр, редактирование и удаление таблиц;
- фильтрация списков таблиц и форм по названию, датам, владельцу и другим признакам;
- создание и редактирование форм с произвольным набором полей;
- предпросмотр формы и отправка ответов;
- автоматическая запись результатов формы в связанную таблицу;
- импорт и экспорт данных;
- просмотр истории действий по таблицам и формам;
- построение статистики по выбранным столбцам таблицы.

3. Качественные требования к решению

Решение должно быть удобным для повторяющейся работы: основные действия размещаются в левом меню, списки открываются без лишних переходов, а фильтры показываются поверх контента и не меняют структуру страницы. Данные пользователя

должны сохраняться после изменений, история действий должна позволять понять, кто и когда изменял таблицы или формы.

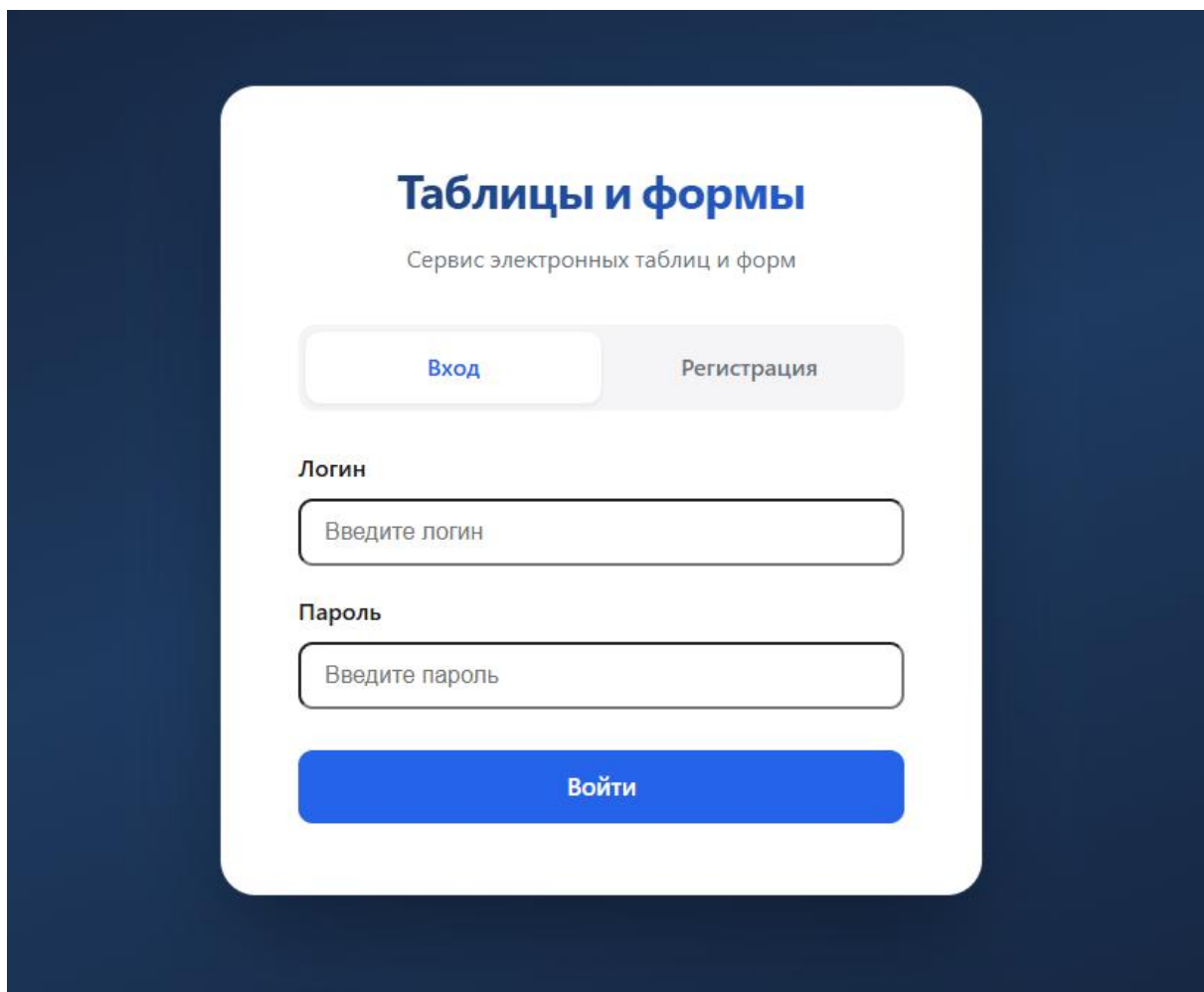
К модели данных предъявляются требования гибкости, компактности и понятности запросов. Таблицы могут иметь разные наборы колонок, формы - разные типы полей, а ответы должны храниться без необходимости заранее создавать отдельную физическую таблицу под каждую форму.

4. Сценарии использования

В проекте представлены основные пользовательские сценарии. Они покрывают полный цикл работы: от входа в приложение до экспорта данных и анализа истории. Ниже приведен обобщенный перечень сценариев, на основе которых формировалась модель данных.

Макеты UI

1. Авторизация (рис .1).



Экран авторизации сервиса «Таблицы и формы». В центре экрана на белом фоне с закругленными углами расположены элементы интерфейса. Вверху — заголовок «Таблицы и формы» и подзаголовок «Сервис электронных таблиц и форм». Ниже — две кнопки: «Вход» (активная, синяя) и «Регистрация» (неактивная, серая). Далее — поля для ввода «Логин» и «Пароль» с подсказками «Введите логин» и «Введите пароль». В самом низу — большая синяя кнопка «Войти».

Рисунок 1. Авторизация.

2. Вкладка с таблицами (Рис. 2).

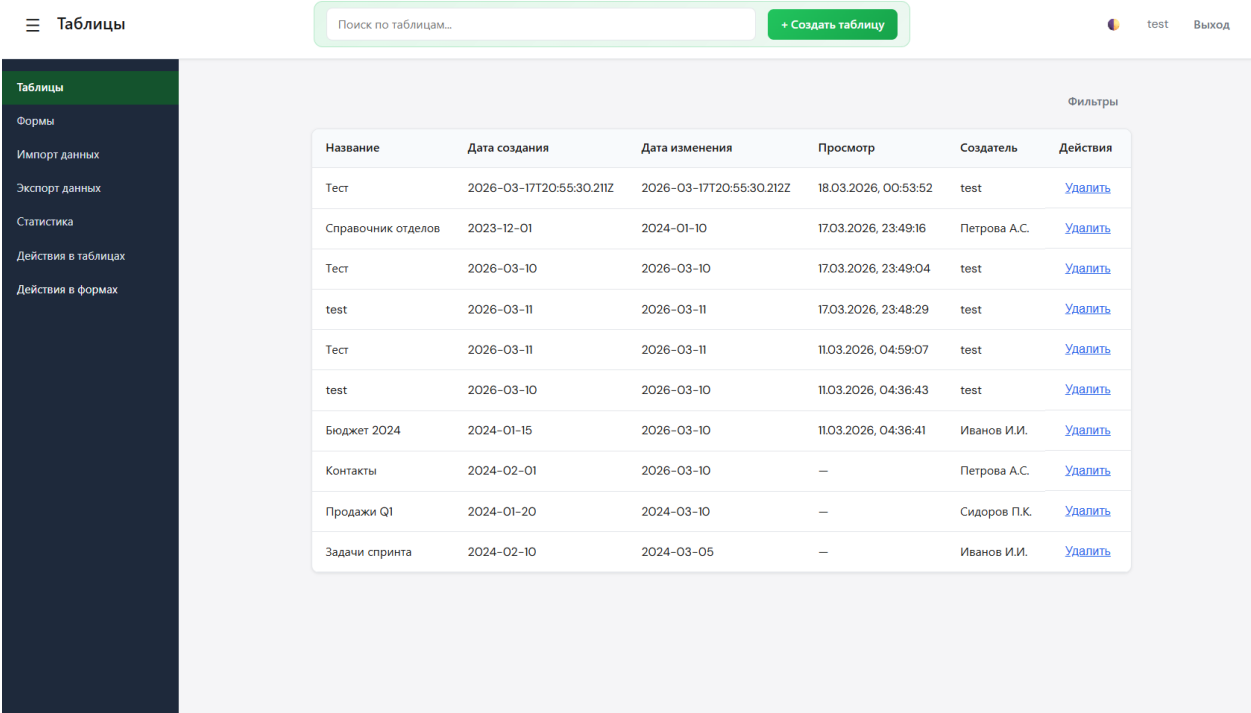


Рисунок 2 Вкладка с таблицами.

3. Вкладка с формами (Рис. 3).

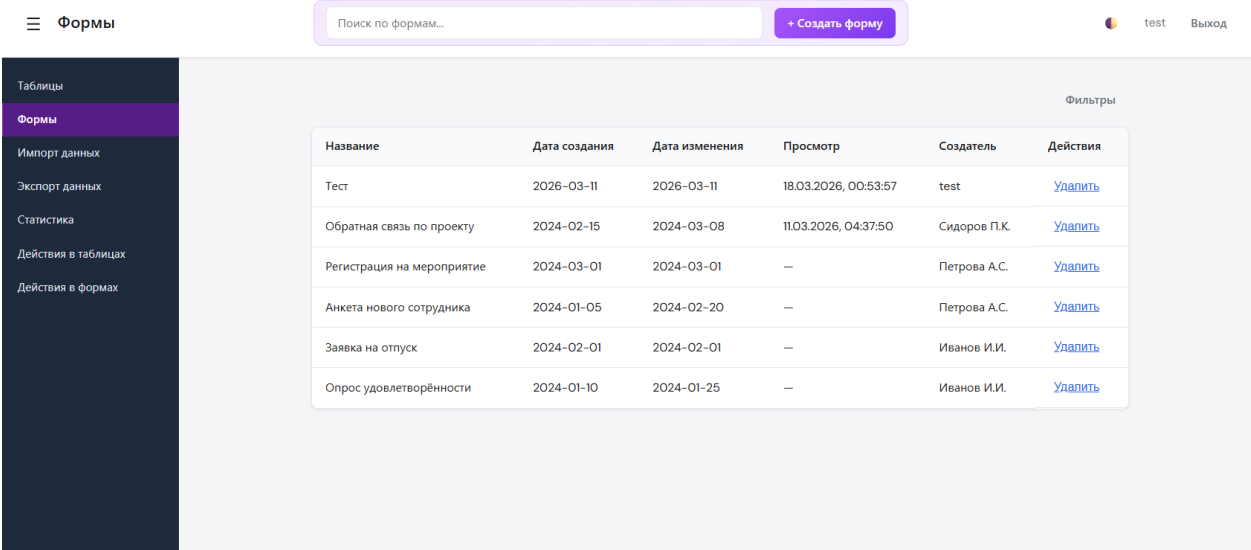


Рисунок 3 Вкладка с формами.

4. Вкладка с статистикой (Рис. 4).

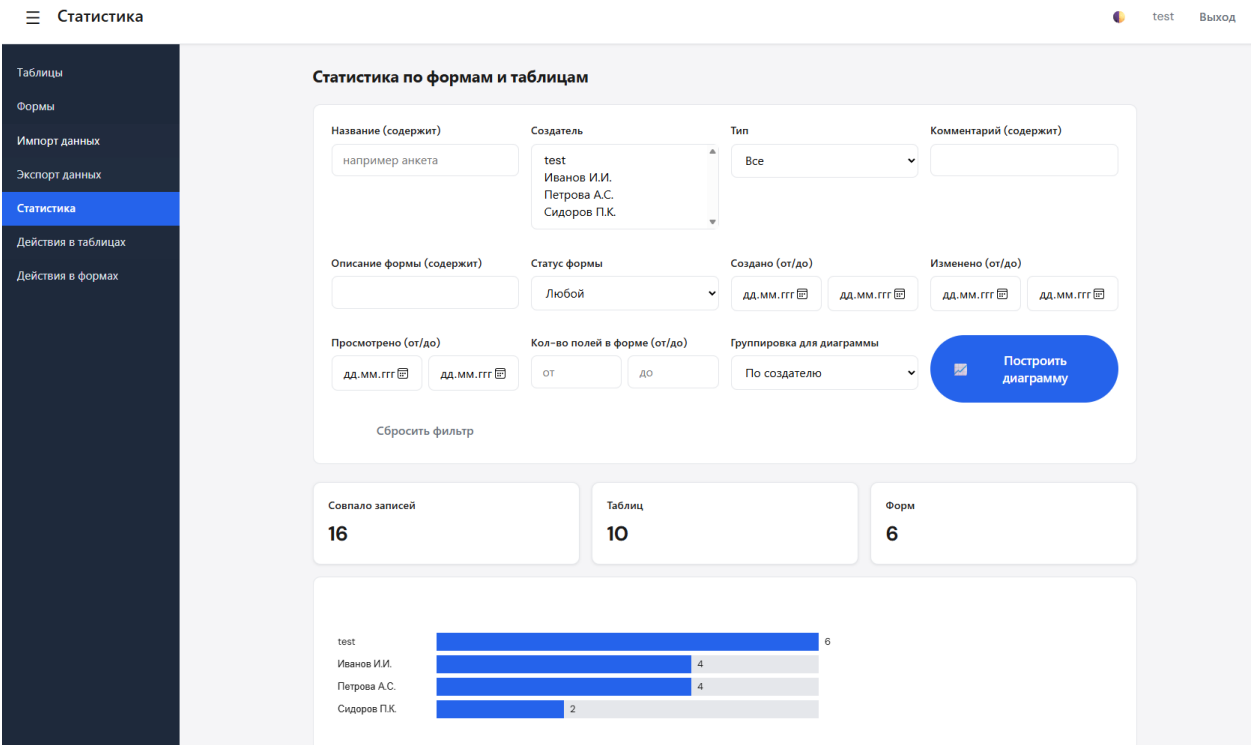


Рисунок 4 Вкладка с статистикой.

5. Вкладка действий в таблицах (Рис. 5).

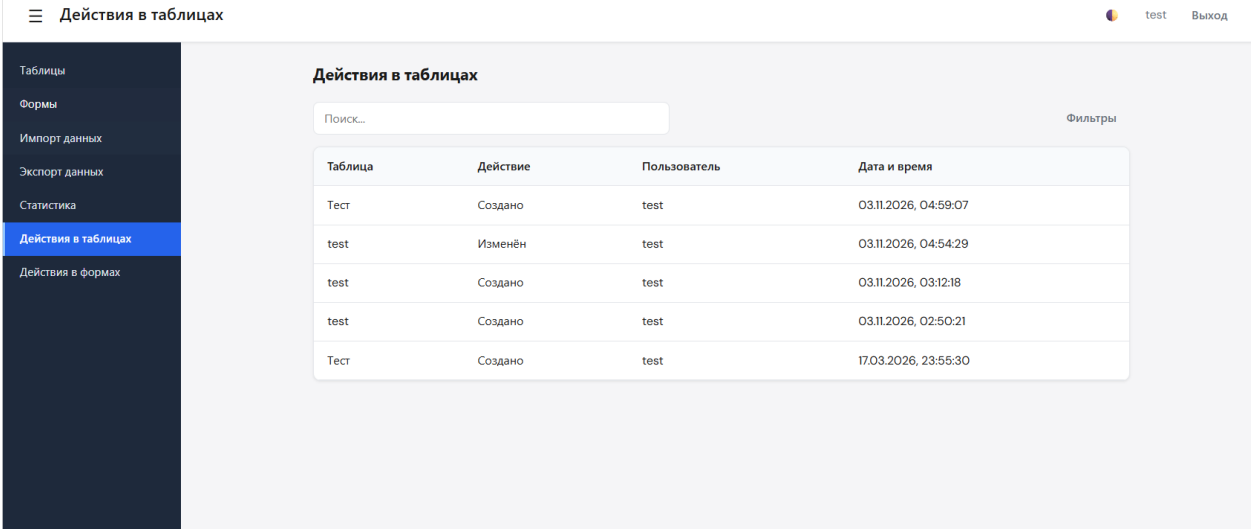


Рисунок 5 Вкладка действий в таблицах.

6. Вкладка действий в формах (Рис. 6).

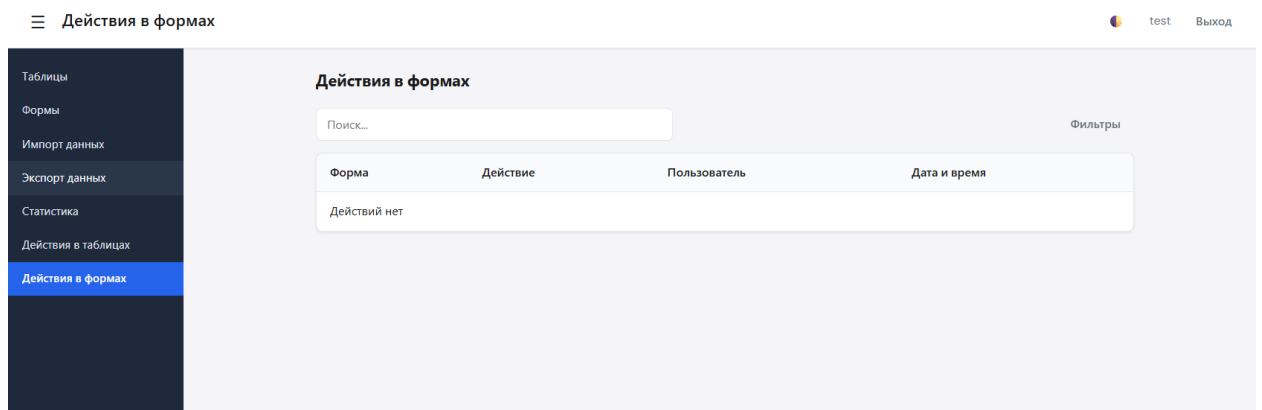


Рисунок 6 Вкладка действий в формах.

Описание сценариев использования

1. Авторизация

Действующее лицо: Пользователь.

Основной сценарий: Пользователь открывает приложение и видит экран авторизации с вкладками «Вход» и «Регистрация». Пользователь выбирает вкладку «Вход». Вводит логин и пароль. Нажимает кнопку «Войти». При корректных данных пользователь попадает в основное приложение на вкладку «Таблицы» (список таблиц).

1.2. Регистрация

Действующее лицо: Новый пользователь.

Основной сценарий: Пользователь открывает приложение, видит экран авторизации. Переключается на вкладку «Регистрация». Вводит новый логин и пароль, подтверждает пароль. Нажимает кнопку «Зарегистрироваться». При успешной регистрации учётная запись сохраняется, пользователь автоматически считается вошедшим. Пользователь попадает на главную страницу приложения (вкладка «Таблицы»).

2.1. Вкладка с таблицами (список таблиц)

Действующее лицо: Пользователь.

Основной сценарий: Пользователь находится в приложении и выбирает в левом меню пункт «Таблицы». В шапке видит поле поиска по таблицам и кнопку «+ Создать

таблицу». Над списком - кнопка «Фильтры» (см. п. 2.2). Ниже отображается таблица со списком таблиц: название, дата и время создания, дата и время изменения, дата и время последнего просмотра, создатель, действия (удаление). При клике по строке (не по кнопке «Удалить») пользователь переходит в режим просмотра/редактирования выбранной таблицы.

2.2. Применение фильтров (для списка таблиц)

Действующее лицо: Пользователь.

Основной сценарий: На вкладке «Таблицы» пользователь при необходимости вводит текст в поле поиска в шапке. Нажимает кнопку «Фильтры» над таблицей - открывается всплывающая панель поверх контента (список не сдвигается). В панели отображаются критерии (вертикальный список кнопок): по дате просмотра (по умолчанию), по названию, по дате создания, по дате изменения, по создателю. Пользователь нажимает один из критериев - он сразу становится активным, список пересчитывается: если в поиске есть текст - отображаются только строки, у которых выбранное поле содержит введенную подстроку (без учёта регистра); если поиск пустой - список сортируется по выбранному полю. Повторное нажатие на «Фильтры» скрывает панель.

3. Окно создания таблицы

Действующее лицо: Пользователь.

Основной сценарий: На вкладке «Таблицы» пользователь нажимает кнопку «+ Создать таблицу». Открывается экран «Создание таблицы» с полем «Название таблицы». Пользователь вводит название и нажимает кнопку «Создать таблицу». Система создаёт новую таблицу с базовой сеткой ячеек и текущими датами создания/изменения. Пользователь автоматически перенаправляется в окно редактирования таблицы.

4. Редактирование таблицы

Действующее лицо: Пользователь.

Основной сценарий: Пользователь открывает таблицу (через клик по строке в списке). На экране таблицы видит: название таблицы (редактируемое поле), область комментария, верхнюю панель с кнопками «Фильтры», «История», «Сохранить», сетку ячеек (строки 1..40, столбцы A..AN). Пользователь редактирует содержимое ячеек (вводит текст/числа). По мере изменения значений таблица сохраняется в локальном хранилище.

При необходимости пользователь нажимает «Сохранить» для фиксации имени и комментария.

5. Удаление таблицы

Действующее лицо: Пользователь.

Основной сценарий: Пользователь открывает вкладку «Таблицы». В списке таблиц находит нужную строку. Нажимает кнопку «Удалить» в колонке «Действия». Появляется диалог подтверждения: «Удалить таблицу? Таблица будет удалена без возможности восстановления. Это действие нельзя отменить». Пользователь нажимает «Удалить». Таблица удаляется из хранилища и пропадает из списка.

6.1. Окно с формами (список форм)

Действующее лицо: Пользователь.

Основной сценарий: Пользователь выбирает в левом меню пункт «Формы». В шапке видит поле поиска по формам и кнопку «+ Создать форму». Над списком - кнопка «Фильтры» (см. п. 6.2). Ниже отображается таблица со списком форм: название, дата и время создания и изменения, просмотр, создатель, действия. При клике по строке открывается экран редактирования формы. Для удаления формы пользователь нажимает «Удалить» в колонке «Действия» и подтверждает в диалоге.

6.2. Применение фильтров (для списка форм)

Действующее лицо: Пользователь.

Основной сценарий: На вкладке «Формы» пользователь при необходимости вводит текст в поле поиска в шапке. Нажимает «Фильтры» - открывается всплывающая панель с критериями (аналогично списку таблиц): по дате просмотра (по умолчанию), по названию, по дате создания, по дате изменения, по создателю. Один клик по критерию сразу применяет фильтрацию по поисковой строке или сортировку при пустом поиске. Повторное нажатие «Фильтры» скрывает панель.

7. Создание формы

Действующее лицо: Пользователь (создатель формы).

Основной сценарий: На вкладке «Формы» пользователь нажимает кнопку «+ Создать форму». Открывается экран конструктора формы. Пользователь задаёт: название формы, описание, набор полей (тип, заголовок, варианты ответов для списков/чекбоксов и

т.п.). Пользователь нажимает «Создать форму». Форма сохраняется, появляется в списке форм и может быть отредактирована или использована для сбора ответов.

8. Редактирование формы

Действующее лицо: Пользователь (владелец или редактор формы).

Основной сценарий: Пользователь открывает форму (клик по строке в списке форм). На экране формы видит: поле с названием формы, поле комментария, вкладки «Редактировать», «Предпросмотр», «Ответы». На вкладке «Редактировать» пользователь: меняет типы полей, редактирует заголовки и варианты ответов, добавляет или удаляет поля. Нажимает «Сохранить», изменения записываются, обновляется дата изменения формы.

9. Предпросмотр форм

Действующее лицо: Пользователь (проверяющий форму).

Основной сценарий: На экране формы пользователь переключается на вкладку «Предпросмотр». Видит форму так, как её увидит респондент (поля ввода, варианты ответа). При необходимости возвращается на вкладку «Редактировать», чтобы поправить текст или структуру.

10. Результаты форм (ответы)

Действующее лицо: Пользователь (аналитик/владелец формы).

Основной сценарий: Пользователь открывает нужную форму. Переходит на вкладку «Ответы». Видит таблицу с ответами: по строкам - отдельные ответы, по столбцам - поля формы. Может просматривать и анализировать собранные данные.

11. Удаление формы

Действующее лицо: Пользователь.

Основной сценарий: Пользователь находится на вкладке «Формы». Находит нужную форму в списке. Нажимает кнопку «Удалить» в колонке «Действия». Появляется диалог подтверждения: «Удалить форму? Форма и ответы будут удалены без возможности восстановления». Пользователь нажимает «Удалить», форма и её ответы удаляются из хранилища.

12. Окно экспорта

Действующее лицо: Пользователь.

Основной сценарий: Пользователь выбирает в левом меню пункт «Экспорт данных». Видит две карточки: «Таблицы» и «Формы» со списками чекбоксов. Отмечает какие именно таблицы и формы нужно выгрузить. Если ничего не отмечено, по умолчанию экспортируются все. В блоке «Формат» выбирает один из вариантов: JSON, CSV или XML. Нажимает кнопку «Скачать экспорт». Приложение формирует файл выбранного формата и предлагает его скачать.

13. Окно импорта

Действующее лицо: Пользователь.

Основной сценарий: Пользователь выбирает в левом меню пункт «Импорт данных». Выбирает формат файла (JSON/CSV/XML) и файл с данными. Подтверждает запуск импорта. Приложение загружает данные и обновляет локальное хранилище (таблицы и формы). После завершения импорта пользователь возвращается к списку таблиц и видит обновлённые данные.

14. Окно статистики

Действующее лицо: Пользователь (аналитик).

Пользователь открывает вкладку «Статистика».

Выбирает источник данных: метаданные всех таблиц и форм или строки одной выбранной таблицы. Настраивает оси и при необходимости многокритериальный фильтр, затем нажимает «Построить диаграмму» — результат отображается на полотне под формой.

Основной сценарий (метаданные — таблицы и формы):

В поле «Источник данных» выбрано «Метаданные (таблицы и формы)».

В «Ось X (категории)» пользователь выбирает, по какому признаку группировать объекты на диаграмме:

владелец;

год создания;

год изменения;

месяц создания (группировка в формате год-месяц);

тип (таблица / форма).

В «Фильтр: тип» задаёт подмножество: все объекты, только таблицы или только формы.

При необходимости добавляет условия фильтра (кнопка «+ Добавить условие»): для каждого условия выбирается «столбец» в логической схеме (название, владелец, даты, тип) и оператор — *содержит / не содержит / равно / не равно / больше / меньше / $\geq$ / $\leq$ / пусто / не пусто* (часть операций применима к числовым и текстовым сравнениям). Несколько условий объединяются по И (должны выполняться все).

Нажимает «Построить диаграмму» — строится столбчатая диаграмма количества объектов по выбранной оси X на отфильтрованном наборе.

Основной сценарий (данные из одной таблицы):

В «Источник данных» выбрано «Данные из таблицы».

Пользователь выбирает таблицу из списка.

Указывает ось X — столбец сетки (буква столбца A, B, C ...), значения которого станут категориями на диаграмме.

При необходимости указывает ось Y (группировка) — второй столбец; тогда для каждой категории по X показывается распределение по значениям столбца Y (составная визуализация).

В блоке условий фильтра задаёт ограничения по ячейкам строк (столбец A..., оператор, значение) — аналогично расширенным операторам; отбираются только строки, удовлетворяющие всем условиям.

Нажимает «Построить диаграмму» — отображается диаграмма по отфильтрованным строкам.

Альтернативный сценарий:

Не выбрана таблица или не задана ось X для режима «данные из таблицы» — приложение выводит подсказку на полотне вместо диаграммы.

15. Окно «Действия в формах»

Действующее лицо: Пользователь (аудит / владелец данных).

Пользователь выбирает в меню пункт «Действия в формах».

Видит общий журнал записей: форма, действие (например, создание, изменение, удаление), пользователь, дата и время.

Слева от таблицы — поле «Поиск...», справа — кнопка «Фильтры» (тот же принцип, что на вкладке «Формы»): всплывающая вертикальная панель с критериями по дате и времени (по умолчанию), по форме, по действию, по пользователю.

Ввод текста в поиск и выбор критерия сразу отфильтровывают журнал по выбранному полю или сортируют список при пустом поиске.

Альтернативный сценарий:

История пуста — в таблице отображается сообщение, что действий нет.

16. Окно «Действия в таблицах»

Действующее лицо: Пользователь (аудит / владелец данных).

Пользователь выбирает в меню пункт «Действия в таблицах».

Видит общий журнал по всем таблицам: название таблицы, действие, пользователь, дата и время. Использует поиск и «Фильтры» с критериями: по дате и времени (по умолчанию), по таблице, по действию, по пользователю — поведение аналогично п. 2.2 и п. 15.

Таким образом можно быстро найти, кто и когда менял конкретную таблицу.

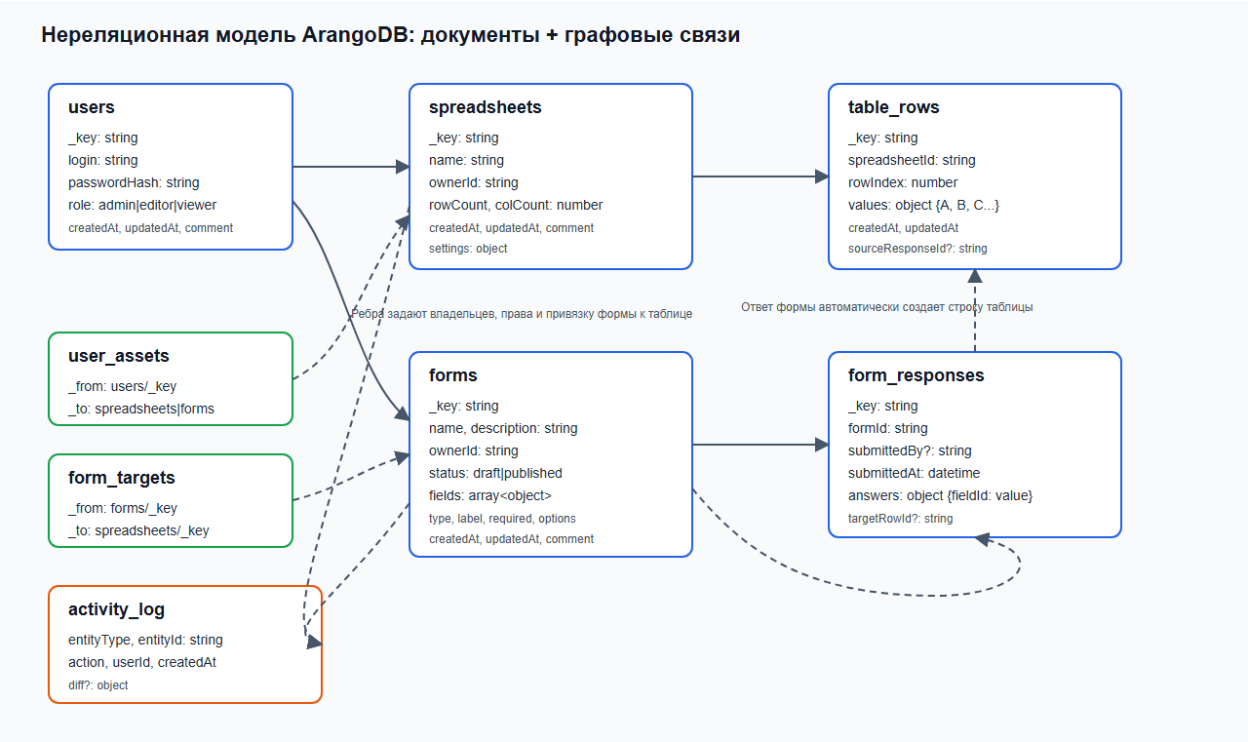
Альтернативный сценарий:

Для таблицы ещё не велась история — соответствующих строк в журнале нет.

5. Модель данных

Нереляционная модель

Используемая СУБД - ArangoDB. Модель сочетает документное хранение JSON и графовые связи. Документы удобны для таблиц, форм и ответов, потому что структура формы и набор колонок могут меняться от объекта к объекту. Ребра удобны для связей "пользователь владеет таблицей/формой" и "форма записывает ответы в таблицу".



Коллекции и назначение

Коллекция	Тип ArangoDB	Назначение
users	document	Учетные записи пользователей и роли для доступа к приложению.
spreadsheets	document	Метаданные электронных таблиц: название, владелец, даты, комментариев, размеры сетки.
table_rows	document	Строки таблиц. Значения ячеек хранятся в объекте <code>values</code> , например <code>{ "A": "Проект", "B": "100" }</code> .
forms	document	Формы и их схема. Поля формы вложены массивом <code>fields</code> , потому что набор вопросов является частью формы.
form_responses	document	Отправленные ответы на формы. Ответы хранятся объектом <code>answers</code> , где ключ - идентификатор поля формы.
activity_log	document	История изменений таблиц и форм с датой, пользователем и типом действия.
user_assets	edge	Связь пользователя с таблицей или формой: владелец, редактор, читатель.
form_targets	edge	Связь формы с таблицей, куда автоматически добавляются ответы.

Поля и оценка размера

Оценка дана для UTF-8. Для строк с русским текстом принято среднее 2 байта на символ. В фактическом объеме учитываются имена JSON-полей, кавычки, служебные символы и системные поля ArangoDB (`_key`, `_id`, `_rev`). Значения округлены, потому что точный размер зависит от конкретных строк и индексов.

Объект	Основные поля	Чистый объем данных	Фактический объем документа
users	<code>_key</code> 16 Б, <code>login</code> 32 Б, <code>passwordHash</code> 60 Б, <code>role</code> 8 Б, даты 48 Б, <code>comment</code> 112 Б	276 Б	520 Б
spreadsheets	<code>_key</code> 16 Б, <code>name</code> 80 Б, <code>ownerId</code> 16 Б, даты 48 Б, <code>comment</code> 256 Б, размеры 8 Б	424 Б	760 Б
table_rows	<code>_key</code> 16 Б, <code>spreadsheetId</code> 16 Б, <code>rowIndex</code> 4 Б, 10 значений по 24 Б, даты 48 Б	324 Б	620 Б
forms	<code>_key</code> 16 Б, <code>name</code> 80 Б, <code>description</code> 256 Б, <code>ownerId</code> 16 Б, <code>status</code> 10 Б, даты 48 Б, <code>comment</code> 256 Б, 5 полей формы примерно по 227 Б	1817 Б	2600 Б
form_responses	<code>_key</code> 16 Б, <code>formId</code> 16 Б, <code>submittedBy</code> 16 Б, <code>submittedAt</code> 24 Б, 5 ответов по 40 Б	272 Б	520 Б
activity_log	<code>_key</code> 16 Б, <code>entityType</code> 10 Б, <code>entityId</code> 16 Б, <code>action</code> 40 Б, <code>userId</code> 16 Б, дата 24 Б	122 Б	260 Б
user_assets	<code>_from</code> 16 Б, <code>_to</code> 16 Б, <code>role</code> 10 Б	42 Б	180 Б
form_targets	<code>_from</code> 16 Б, <code>_to</code> 16 Б	32 Б	180 Б

Оценка объема информации

Введем переменную N - количество электронных таблиц. Для единой формулы принимается демонстрационный профиль:

- на 1 таблицу приходится 1 форма;
- в каждой таблице 20 непустых строк и 10 логических колонок;
- у каждой формы 5 полей и 50 ответов;
- на каждую таблицу и форму вместе приходится 10 записей истории;
- 1 пользователь приходится на 5 таблиц, то есть $0.2N$ пользователей;
- на каждую таблицу приходится 2 ребра `user_assets` и 1 ребро `form_targets`.

Чистый объем:

$$V_clean_noSQL(N) = 0.2N * 276 + N * 424 + 20N * 324 + N * 1817 + 50N * 272 + 10N * 122 + N * 42 + N * 32 = 23712.2N \text{ байт}$$

Фактический объем:

$$V_fact_noSQL(N) = 0.2N * 520 + N * 760 + 20N * 620 + N * 2600 + 50N * 520 + 10N * 260 + 2N * 180 + N * 180 = 45004N \text{ байт}$$

С учетом служебной информации базы, первичных индексов и небольшого постоянного системного запаса можно записать:

$$V_noSQL(N) = 45004N + 1024 \text{ байт}$$

Например, для $N = 10$ таблиц потребуется примерно:

$$V_noSQL(10) = 451064 \text{ байт} \approx 441 \text{ КиБ}$$

Избыточность данных

Избыточность определим как отношение фактического объема к чистому объему:

$$R_noSQL(N) = (45004N + 1024) / (23712.2N)$$

При $N = 10$:

$$R_noSQL(10) = 451064 / 237122 \approx 1.90$$

Избыточность возникает из-за имен JSON-полей, системных полей ArangoDB, ребер графа, дат обновления и повторяющихся идентификаторов связей. При росте N влияние постоянного слагаемого 1024 уменьшается, и коэффициент стремится к $45004 / 23712.2 \approx 1.90$.

Направление роста модели

Что увеличивается	Как растет объем
Количество пользователей <code>U</code>	Линейно: $O(U)$, пользовательские документы и ребра доступа.
Количество таблиц <code>T</code>	Линейно по метаданным: $O(T)$, но фактически доминируют строки таблиц.
Количество строк <code>R</code> и заполненных колонок <code>C</code>	$O(T * R * C)$ по значениям в <code>table_rows.values</code> .
Количество форм <code>F</code>	$O(F * Q)$, где <code>Q</code> - число полей формы.
Количество ответов <code>A</code>	$O(A * Q)$, потому что каждый ответ хранит значения по полям формы.
Количество действий в истории <code>L</code>	$O(L)$, каждая операция добавляет документ в <code>activity_log</code> .

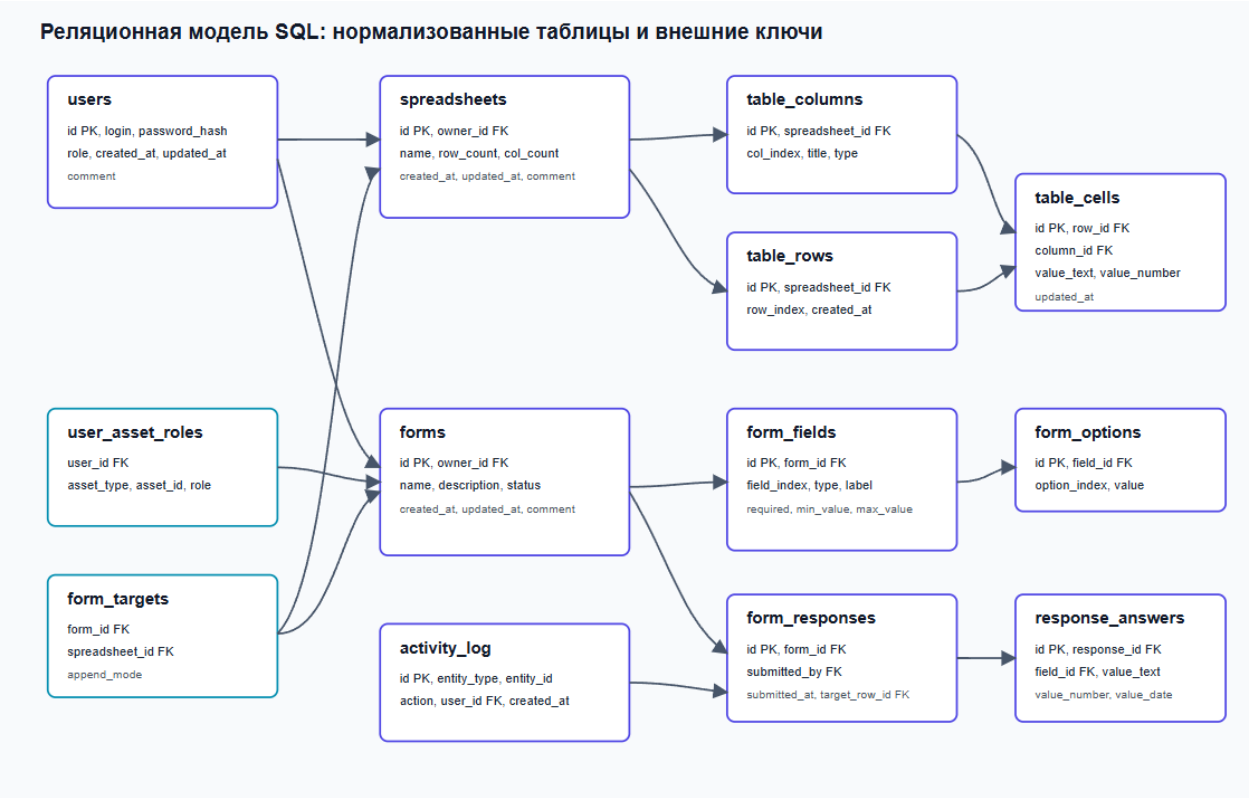
Примеры данных: Таблица

```
{
  "_key": "tbl_budget_2024",
  "name": "Бюджет 2024",
  "ownerId": "u_ivanov",
  "rowCount": 40,
  "colCount": 40,
  "createdAt": "2024-01-15T10:00:00Z",
  "updatedAt": "2024-03-01T11:30:00Z",
  "comment": "",
  "settings": {
    "frozenRows": 1,
    "defaultColumnWidth": 120
  }
}
```

```
{
  "_key": "row_budget_2024_2",
  "spreadsheetId": "tbl_budget_2024",
  "rowIndex": 2,
  "values": {
    "A": "Маркетинг",
    "B": "100",
    "C": "95",
    "D": "-5"
  },
  "createdAt": "2024-01-15T10:01:00Z",
  "updatedAt": "2024-03-01T11:30:00Z"
}
```

Реляционная модель

SQL-аналог нормализует вложенные структуры в отдельные таблицы. Это уменьшает неоднозначность схемы, но увеличивает число таблиц, внешних ключей и соединений при чтении формы, ответов и строк электронных таблиц.



Таблицы и назначение

Таблица	Назначение
users	Пользователи и роли.
spreadsheets	Метаданные электронных таблиц.
table_columns	Описание колонок таблицы.
table_rows	Строки таблицы без значений ячеек.
table_cells	Значения отдельных ячеек.
forms	Метаданные форм.
form_fields	Поля формы.
form_options	Варианты ответов для select , radio , checkbox .
form_responses	Факт отправки формы.
response_answers	Ответы на отдельные поля формы.
activity_log	История действий.
user_asset_roles	Права пользователя на таблицы и формы.
form_targets	Таблица, в которую сохраняются ответы формы.

Поля и оценка размера

Объект SQL	Основные поля	Чистый объем данных	Фактический объем строки
users	id, login, password_hash, role, даты, comment	276 Б	470 Б
spreadsheets	id, owner_id, name, даты, comment, размеры	424 Б	430 Б
table_columns	id, spreadsheet_id, col_index, title, type	70 Б	180 Б
table_rows	id, spreadsheet_id, row_index, даты	44 Б	160 Б
table_cells	id, row_id, column_id, значение, дата	64 Б	190 Б
forms	id, owner_id, name, description, status, даты, comment	682 Б	650 Б
form_fields	id, form_id, field_index, type, label, required, ограничения	107 Б	260 Б
form_options	id, field_id, option_index, value	60 Б	180 Б
form_responses	id, form_id, submitted_by, дата, target_row_id	72 Б	220 Б
response_answers	id, response_id, field_id, значение	56 Б	210 Б
activity_log	id, тип сущности, id сущности, действие, пользователь, дата	122 Б	220 Б
user_asset_roles	user_id, asset_type, asset_id, role	46 Б	170 Б
form_targets	form_id, spreadsheet_id, режим добавления	36 Б	160 Б

В SQL часть строк может иметь меньший физический размер, чем JSON-документы, но для таблиц и ответов появляется много отдельных строк: каждая ячейка и каждый ответ на поле становятся отдельной записью.

Оценка объема информации

Используем тот же демонстрационный профиль и ту же переменную N:

- 1 таблица и 1 форма на единицу N;
- 20 строк, 10 колонок и 200 непустых ячеек на таблицу;
- 5 полей формы, 10 вариантов ответов суммарно;
- 50 отправок формы и 250 отдельных ответов на поля;
- 10 записей истории;
- 0.2 пользователя, 2 записи прав и 1 связь формы с таблицей на единицу N.

Чистый объем SQL-модели:

$$V_clean_SQL(N) = 0.2N * 276 + N * 424 + 10N * 70 + 20N * 44 + 200N * 64 + N * 682 + 5N * 107 + 10N * 60 + 50N * 72 + 250N * 56 + 10N * 122 + 2N * 46 + N * 36 = 35624.2N \text{ байт}$$

Фактический объем SQL-модели:

$$V_fact_SQL(N) = 0.2N * 470 + N * 430 + 10N * 180 + 20N * 160 + 200N * 190 + N * 650 + 5N * 260 + 10N * 180 + 50N * 220 + 250N * 210 + 10N * 220 + 2N * 170 + N * 160 = 113474N \text{ байт}$$

С учетом системных страниц, индексов и служебных структур:

$$V_SQL(N) = 113474N + 2048 \text{ байт}$$

Например, для $N = 10$:

$$V_SQL(10) = 1136788 \text{ байт} \approx 1.08 \text{ МиБ}$$

Избыточность данных

$$R_SQL(N) = (113474N + 2048) / (35624.2N)$$

При $N = 10$:

$$R_SQL(10) = 1136788 / 356242 \approx 3.19$$

Избыточность выше из-за большого количества внешних ключей, индексов и строк для ячеек/ответов. При этом SQL-модель лучше контролирует типы и целостность данных.

Направление роста модели

Что увеличивается	Как растёт объем
Пользователи	$O(U)$ плюс записи в <code>user_asset_roles</code> .
Таблицы	$O(T)$ по метаданным, $O(T * C)$ по колонкам.
Ячейки	$O(T * R * C)$ и это главный источник роста.
Формы	$O(F * Q + F * O)$, где Q - поля, O - варианты ответа.
Ответы	$O(A * Q)$, потому что каждый ответ на поле хранится отдельной строкой.
История	$O(L)$.

Примеры данных: Таблица

```
INSERT INTO spreadsheets (id, owner_id, name, row_count, col_count, created_at, updated_at, comment)
VALUES ('tbl_budget_2024', 'u_ivanov', 'Бюджет 2024', 40, 40,
      '2024-01-15T10:00:00Z', '2024-03-01T11:30:00Z', '');

INSERT INTO table_columns (id, spreadsheet_id, col_index, title, type)
VALUES
  ('col_budget_a', 'tbl_budget_2024', 1, 'Проект', 'text'),
  ('col_budget_b', 'tbl_budget_2024', 2, 'План', 'number'),
  ('col_budget_c', 'tbl_budget_2024', 3, 'Факт', 'number');

INSERT INTO table_rows (id, spreadsheet_id, row_index, created_at)
VALUES ('row_budget_2', 'tbl_budget_2024', 2, '2024-01-15T10:01:00Z');

INSERT INTO table_cells (row_id, column_id, value_text, value_number, updated_at)
VALUES
  ('row_budget_2', 'col_budget_a', 'Маркетинг', NULL, '2024-03-01T11:30:00Z'),
  ('row_budget_2', 'col_budget_b', NULL, 100, '2024-03-01T11:30:00Z'),
  ('row_budget_2', 'col_budget_c', NULL, 95, '2024-03-01T11:30:00Z');
```

Вывод

В выбранной предметной области SQL-модель занимает больше места, потому что электронные таблицы и ответы форм распадаются на большое число строк `table_cells` и `response_answers`. В ArangoDB строка таблицы и ответ формы хранятся компактнее как JSON-документы с вложенным объектом значений.

Если таблицы станут очень широкими и большинство ячеек будет пустым, преимущество ArangoDB сохранится при хранении только непустых ключей в `values`. Если потребуется строгая типизация каждой колонки и сложные транзакционные ограничения, SQL-модель может стать удобнее, но объем и число соединений останутся выше.

6. Разработанное приложение

Краткое описание

Back-end представляет собой приложение, реализующее API для работы с пользователями, таблицами, формами, ответами, историей действий, импортом, экспортом и статистикой. Данные приложения хранятся в нереляционной СУБД ArangoDB. Работа с базой данных выполняется через коллекции документов и AQL-запросы.

Импорт и экспорт данных реализованы следующим образом:

1. Данные приложения выгружаются из ArangoDB в промежуточный формат, содержащий информацию о пользователях, таблицах, строках таблиц, формах, ответах, связях доступа и истории действий.
2. При импорте данные из промежуточного формата считываются приложением и записываются обратно в соответствующие коллекции ArangoDB.

Front-end представляет собой web-приложение, которое использует API back-end приложения и отображает данные удобным образом для пользователя. Через интерфейс пользователь может создавать и редактировать таблицы, создавать формы, отправлять и просматривать ответы, применять фильтры, строить статистику, выполнять импорт и экспорт данных.

Использованные технологии

БД: ArangoDB

Язык запросов: AQL

Back-end: Python

Front-end: HTML, CSS, JavaScript

Экраны приложения и переходы между ними отображены на рисунке 7.



Рисунок 7 Схема экранов приложения

Ссылка на Приложение

Ссылка на github: <https://github.com/moevm/nsql1h26-table>

7. Вывод

Для данного проекта лучше подходит NoSQL-модель в ArangoDB. Она компактнее для выбранного профиля данных, проще хранит гибкие структуры форм и таблиц, позволяет одним AQL-запросом получать документ вместе со связанными строками или ответами, а графовые коллекции естественно описывают права пользователей и привязку формы к таблице. SQL-модель остается полезной как контрольная альтернатива: она строже по типам и внешним ключам, но в нашем приложении дает больший объем хранения и больше соединений для типовых сценариев.

Таким образом, выбранная модель соответствует ключевым сценариям приложения: созданию таблиц с изменяемыми колонками, созданию форм с произвольным набором полей, записи ответов в таблицы, построению статистики и ведению истории действий.

Недостатки и пути улучшения

Текущая модель предполагает хранение строк таблицы как документов с объектом значений. При большом количестве строк и активной фильтрации по произвольным столбцам может потребоваться дополнительная оптимизация индексов и запросов.

Также дальнейшего развития требует механизм импорта и экспорта: необходимо предусмотреть проверку структуры входных данных, обработку конфликтов идентификаторов и понятные сообщения об ошибках для пользователя.

Будущее развитие решения

В дальнейшем приложение можно развивать в следующих направлениях:

- добавить расширенные права доступа к таблицам и формам;
- добавить более гибкую настройку типов полей формы;
- расширить модуль статистики и визуализации данных;

8. Приложение

Документация по сборке и разворачиванию приложения

- Скачать проект из репозитория.
- Установить зависимости проекта.
- Запустить ArangoDB.
- Настроить подключение к базе данных.
- Запустить back-end приложение.
- Запустить front-end приложение.
- Открыть приложение в браузере.

9. Используемая литература

1. Документация ArangoDB: <https://docs.arangodb.com/>
2. Документация AQL: <https://docs.arangodb.com/stable/aql/>
3. Документация по JavaScript: <https://developer.mozilla.org/ru/docs/Web/JavaScript>
4. Документация по HTML: <https://developer.mozilla.org/ru/docs/Web/HTML>
5. Документация по CSS: <https://developer.mozilla.org/ru/docs/Web/CSS>

10. Примеры запросов

Ниже приведены AQL-запросы, которые реализуют основные сценарии использования. Под n понимается число документов в основной коллекции сценария, под r - число строк выбранной таблицы, под a - число ответов формы.

Авторизация

```
FOR u IN users
  FILTER LOWER(u.login) == LOWER(@login)
  LIMIT 1
  RETURN {
    _key: u._key,
    login: u.login,
    passwordHash: u.passwordHash,
    role: u.role
  }
```

Количество запросов: 1. Коллекции: users. Сложность при индексе по login - $O(\log n)$, без индекса - $O(n)$.

Просмотр списка таблиц с многокритериальной фильтрацией

```
FOR t IN spreadsheets
  FILTER @name == null OR CONTAINS(LOWER(t.name), LOWER(@name))
  FILTER @ownerId == null OR t.ownerId == @ownerId
  FILTER @createdFrom == null OR t.createdAt >= @createdFrom
  FILTER @createdTo == null OR t.createdAt <= @createdTo
  FILTER @updatedFrom == null OR t.updatedAt >= @updatedFrom
  FILTER @updatedTo == null OR t.updatedAt <= @updatedTo
  SORT t.updatedAt DESC
  LIMIT @offset, @limit
  RETURN t
```

Количество запросов: 1 для списка, дополнительно 1 при необходимости отдельного подсчета общего количества. Коллекции: spreadsheets. Сложность - $O(\log n + k)$ при индексах по датам и владельцу, где k - число найденных объектов; подстрочный поиск по названию без ArangoSearch - $O(n)$.

Открытие таблицы

```
LET table = DOCUMENT(spreadsheets, @tableKey)
LET rows = (
  FOR r IN table_rows
    FILTER r.spreadsheetId == @tableKey
    SORT r.rowIndex ASC
    LIMIT @offset, @limit
```

```

    RETURN r
)
RETURN { table, rows }

```

Количество запросов: 1. Коллекции: spreadsheets, table_rows. Сложность - $O(\log r + \text{pageSize})$ при индексе table_rows.spreadsheetId.

Изменение строки таблицы и запись истории

```

LET oldRow = FIRST(
  FOR r IN table_rows
    FILTER r.spreadsheetId == @tableKey AND r.rowIndex == @rowIndex
  LIMIT 1
  RETURN r
)
LET savedRow = UPSERT { spreadsheetId: @tableKey, rowIndex:
@rowIndex }
  INSERT {
    spreadsheetId: @tableKey,
    rowIndex: @rowIndex,
    values: @values,
    createdAt: DATE_ISO8601(DATE_NOW()),
    updatedAt: DATE_ISO8601(DATE_NOW())
  }
  UPDATE {
    values: MERGE(oldRow.values, @values),
    updatedAt: DATE_ISO8601(DATE_NOW())
  }
  IN table_rows
  RETURN NEW
INSERT {
  entityType: "spreadsheet",
  entityId: @tableKey,
  action: "row_updated",
  userId: @userKey,
  createdAt: DATE_ISO8601(DATE_NOW()),
  diff: { rowIndex: @rowIndex, values: @values }
} INTO activity_log
RETURN savedRow[0]

```

Количество запросов: 1 транзакционный AQL-запрос. Коллекции: table_rows, activity_log. Сложность - $O(\log r)$ при уникальном индексе (spreadsheetId, rowIndex).

Создание формы

```

LET form = (
  INSERT {
    name: @name,
    description: @description,
    ownerId: @userKey,
    status: "draft",
    fields: @fields,
    createdAt: DATE_ISO8601(DATE_NOW()),
  }
)

```

```

        updatedAt: DATE_ISO8601 (DATE_NOW()),
        comment: @comment
    } INTO forms
    RETURN NEW
)
INSERT {
    _from: CONCAT("users/", @userKey),
    _to: form[0]._id,
    role: "owner"
} INTO user_assets
RETURN form[0]

```

Количество запросов: 1. Коллекции: forms, user_assets. Сложность - $O(1 + q)$, где q - число полей формы.